中图分类号：TN492

学号：ZY2402113

北京航空航天大學

专业硕士学位文献综述

面向端侧NPU的AI编译器研究综述

作者姓名 蒋一平

学科专业 电子信息

指导教师 李洪革教授，李丽副教授

培养院系 电子信息工程学院

2026年10月

摘要

本综述围绕面向RISC-V异构片上系统的AI编译器研究方向，系统梳理了国内外在计算图表示、图级优化、张量程序调度、静态存储规划及软硬件协同编译等方面的研究进展。随着深度神经网络逐步应用于嵌入式视觉、工业检测和智能终端，模型计算量、存储需求与端侧硬件资源之间的矛盾日益突出。AI编译器通过在部署前完成模型分析、数据布局变换、算子映射和代码生成，将上层网络描述转换为目标处理器或专用加速器能够执行的程序，是实现算法与硬件有效衔接的重要工具。

在编译体系方面，多层次中间表示和算法与调度分离的方法为不同硬件后端提供了统一的优化基础；在优化技术方面，计算图重写、算子融合、自动调度以及生命周期分析分别从计算结构、执行方式和数据存储等角度改善推理效率。针对存储资源受限的微控制器和异构SoC，近年来的研究进一步关注静态代码生成、跨层分块、数据搬运规划及轻量运行支持，逐步形成面向资源约束的端到端编译方法。

通过对相关文献的比较可以看出，面向宏指令NPU的优化空间与通用CPU、GPU存在差异，其可控制的重点往往是算子分解、指令序列、张量布局以及存储分配。针对RISC-V处理器与NPU协同构成的FPGA原型系统，后续研究需要在已有离线编译与板端验证基础上，进一步建立包含执行顺序、存储别名和硬件约束的统一模型，研究存储规划与图调度的协同优化，并通过消融实验、数值验证和硬件测量评价方法的有效性。

关键词：AI编译器；RISC-V；NPU；离线编译；存储优化；软硬件协同

Abstract

This review investigates AI compilers for RISC-V heterogeneous systems-on-chip, with a focus on computational graph representations, graph transformations, tensor scheduling, static memory planning, and hardware-software co-design. Deploying deep neural networks on embedded devices requires bridging the gap between model requirements and limited computing and memory resources. Ahead-of-time compilation addresses this challenge by translating model descriptions into target-specific code or accelerator command streams before deployment.

Existing research has established multi-level intermediate representations, operator fusion, automatic scheduling, and memory-aware deployment techniques. Recent work on microcontrollers and heterogeneous edge devices further explores cross-layer tiling, explicit data movement, and code generation with minimal runtime support. These approaches demonstrate that the compilation strategy must account for both the model topology and the capabilities exposed by the target hardware.

For macro-instruction NPUs integrated into RISC-V FPGA prototypes, promising research directions include coordinated tensor lifetime analysis, branch scheduling, and layout-aware memory allocation. Carefully scoped instruction extensions may provide additional opportunities when supported by hardware measurements. Such techniques should be evaluated through numerical validation, ablation studies, and reproducible end-to-end experiments, rather than deployment functionality alone.

Keywords: AI compiler; RISC-V; NPU; ahead-of-time compilation; memory optimization; hardware-software co-design

目录

1.引言 1
2.文献概述 1
3.基本研究现状与发展趋势 3
3.1 多层次中间表示与端到端编译 3
3.2 计算图重写与算子融合 4
3.3 张量程序自动调度与代价模型 5
3.4 面向资源受限设备的静态存储优化 5
3.5 数据流建模与软硬件协同编译 7
3.6 数值一致性与编译结果验证 8
3.7 国内外研究比较与发展趋势 8
4.结论 10
5.主要参考文献 11
附录 文献来源与等级核对 13

1.引言

近年来，深度学习技术在图像识别、目标检测和语音处理等领域获得了广泛应用。随着应用场景由云端逐步向嵌入式设备延伸，神经网络的部署环境呈现出算力有限、存储容量较小、功耗预算严格和软件环境多样等特点。对于此类设备，仅依靠增加计算单元数量难以持续提高系统效率，模型的执行顺序、数据组织方式及访存行为同样会对实际性能产生显著影响。因此，在算法与硬件之间构建有效的编译与优化机制，已成为端侧智能系统研究的重要组成部分。

与传统编译器主要面向通用程序不同，AI编译器的输入通常包含计算图、张量形状、算子属性和模型参数，其优化对象既包括图结构，也包括具体算子的循环、数据布局和硬件映射。Li等人对深度学习编译器进行了系统总结，指出高层计算图优化与底层算子实现共同构成编译系统的核心内容[1]。这一分类表明，AI编译器的研究价值并不局限于模型格式转换，而在于利用模型语义与硬件特性改善执行效率。

面向RISC-V与NPU组成的异构SoC，编译器还承担了软件模型与专用指令之间的转换任务。RISC-V处理器可以负责系统控制、数据准备和执行启动，NPU负责适合其计算结构的张量运算。若模型结构、输入形状和目标硬件在部署前已经确定，则可以提前生成固定的执行计划和存储映射，减少设备运行时的图解析与动态调度需求。这类方法适合裸机或轻量操作系统环境，但仍然需要程序装载、地址配置、同步和结果读取等基本支持，不能将离线编译等同于完全没有运行时软件。

本文以端侧静态推理为主要对象，首先概述AI编译器的技术体系，随后按照中间表示、图级优化、自动调度、存储优化和软硬件协同等方向分析代表性工作，最后结合宏指令NPU的特点讨论值得进一步研究的问题。文献选择以正式发表于CCF推荐A类、B类会议和期刊的研究为主，兼顾基础工作与2024—2025年的相关进展，不以项目宣传材料代替学术依据。

2.文献概述

AI编译器的发展与深度学习计算模式及硬件平台的演进密切相关。早期部署方式较多依赖人工优化的算子库，框架根据网络结构调用对应实现。随着网络结构和加速器类型增多，为每种算子、数据形状及硬件组合分别编写高效实现的成本逐

渐增加。2018年，Chen等人提出TVM，将图级优化、张量计算表示与针对硬件的代码生成相结合，为跨平台部署提供了端到端编译框架[2]。此后，相关研究逐渐从单算子优化扩展至跨算子协同、异构映射和全图资源规划。

在编译基础设施方面，MLIR以多层次中间表示支持领域专用语义的表达与逐级降低，使编译器能够在不同抽象层实施相应变换[3]。在方法发展方面，Halide提出将算法描述与执行调度分离，为局部性、并行性和重计算之间的权衡提供了清晰的表达方式[4]。上述工作分别从基础设施和程序表达角度，为AI编译器中“计算内容”与“执行策略”的分离奠定了基础。

对于嵌入式平台，研究关注点与服务器存在区别。服务器通常具有较大的内存空间和较强的运行时支持，而微控制器或专用SoC常常需要显式管理缓冲区和数据搬运。编译器既要生成目标硬件可执行的程序，也要确保中间张量、模型参数与临时工作区能够被实际存储资源容纳。因此，本文将静态存储规划与端侧部署单独作为研究重点，并将其与图优化和指令生成联系起来进行讨论。

需要说明的是，提前编译、算子库调用与运行时系统是不同维度的概念。提前编译的产物可以是机器码、静态C程序、预先确定的算子调用序列，也可以是专用NPU命令流；这些产物仍可能调用已有算子库或依赖轻量执行支持。相应地，模型文件也未必是可以直接执行的指令镜像。比较不同编译工具时，应具体考察图解析、内存规划、算子选择和执行调度分别发生在部署前还是设备运行阶段。

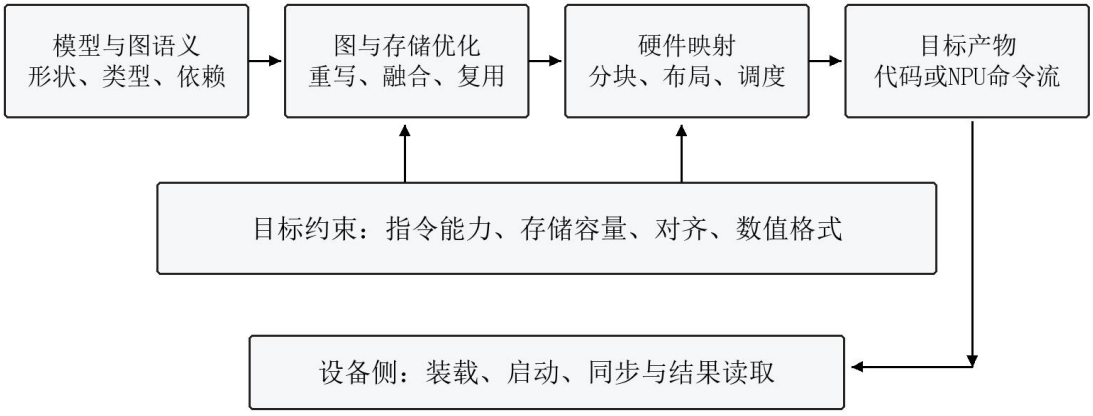


图1 AI编译器的主要优化层次与端侧部署路径

3.基本研究现状与发展趋势

3.1 多层次中间表示与端到端编译

中间表示是连接模型前端、优化过程和硬件后端的基础。高层表示侧重描述算子语义、张量维度及数据依赖，低层表示则逐步引入循环结构、存储空间、并行方式和指令约束。不同层次保留的信息不同，适合实施的优化也不同。如果过早将计算图转换为固定指令序列，可能丢失跨算子分析所需的结构信息；如果始终停留在高层图结构，则又难以准确表达硬件的存储布局和执行限制。

TVM通过连接计算图优化与张量程序生成，使模型部署不再完全依赖为每种网络手工配置的执行过程。对于新型加速器，这一思想的重要性在于：编译器需要同时具有模型级视角和硬件级视角。只有模型级描述而缺少目标约束，可能生成无法执行的计划；只有指令模板而缺少图语义，又容易使优化局限于局部参数调整。面向特定NPU的小型编译器可以借鉴其分层思路，而不必完整复制通用框架的全部功能。

MLIR并不规定所有领域必须采用同一种中间表示，而是通过方言机制表达不同领域的操作和类型，并利用逐级转换将高层语义映射到低层执行形式。这有利于在优化阶段保留形状、索引和存储语义，但同时要求明确各层之间的合法化条件。对于专用NPU，可以分别建立图层、执行计划层和指令层，并将算子支持范围、地址对齐和数值格式作为转换的显式约束。

2023年，Feng等人提出TensorIR，围绕张量化计算建立可组合的程序表示与变换机制，支持将计算结构与硬件张量指令相匹配[5]。与只描述标量循环的表示相比，这类方法能够保留张量计算块的语义，为自动变换和硬件映射提供依据。其启示是，编译器应在适当层次表达硬件可以独立执行的计算单元，而不是在前端就将所有高层结构展开为缺乏语义的指令列表。

不过，中间表示的复杂程度应与目标平台的实际需求相适应。对于仅支持静态形状和有限算子集合的NPU，在已有结构化图表示中补充张量生命周期、物理布局、别名关系及执行约束，可能比直接迁移完整编译基础设施更具可行性。判断表示设计是否有效，应考察其是否支持明确的优化和正确性检查，而不能仅以表示层次数量或采用的框架名称衡量。

3.2 计算图重写与算子融合

计算图重写通过替换等价子图、传播常量和消除冗余操作，改变网络的执行结构。2019年，Jia等人提出TASO，将图替换规则的自动生成与验证结合起来，减少对人工编写优化规则的依赖[6]。这类工作说明，图优化的关键不仅是发现可能减少计算量的替换，还需要保证替换前后的语义关系成立。对于定点NPU，还必须进一步考虑溢出、饱和、舍入及量化位置，因为实数意义上的代数等价并不总能保证有限位宽计算结果一致。

算子融合是图级优化的重要形式，其目标通常包括减少中间数据写回、降低执行启动开销和改善数据局部性。2021年，Niu等人提出DNNFusion，将面向融合的图重写与算子融合决策相结合，以发掘更大的跨算子优化空间[7]。从方法上看，融合不能仅依据算子在图中相邻这一条件，还需要分析迭代关系、数据依赖和实现代价。融合范围过大可能增加临时存储需求，或降低硬件并行度，因此需要结合目标平台评估。

对于宏指令NPU，应区分图结构合并、指令合并和数据路径融合。将卷积与激活记录在同一个执行计划条目中，仅表明软件将它们作为关联操作处理；如果后端仍生成独立指令并将卷积结果写入存储器，则中间访存并未消失。真正的数据路径融合需要硬件能够直接对计算结果执行后处理，或者支持将中间结果保留在可复用的片上缓冲中。因此，评价融合效果时应同时统计指令数量、张量写回次数及访问字节数。

数据布局也会影响图优化的适用范围。以通道拼接为例，若各分支输出能够提前写入最终张量的不同存储区域，则可以省去独立拼接拷贝。但这一方法要求生产者支持相应输出地址或步长，且布局满足目标通道对齐规则。对于多消费者张量，提前改变布局还可能影响其他消费者。因此，消除拼接操作不能脱离别名关系、依赖分析和物理布局检查单独进行。

总体来看，图级优化研究正在由局部模式匹配向包含执行代价和正确性条件的综合决策发展。对于算子覆盖范围有限的原型NPU，优先研究少量出现频繁、语义明确且可以测量收益的模式，能够形成较为清晰的研究问题。与增加大量尚未验证的融合规则相比，说明规则的合法条件、适用范围及失效原因，更有助于形成可信的技术结论。

3.3 张量程序自动调度与代价模型

同一算子可以采用不同的循环顺序、分块尺寸和并行映射，其性能往往存在较大差异。Halide通过分离算法与调度，使这些实现策略能够在不改变算法定义的情况下被描述和调整。该思想被后续张量编译研究广泛借鉴，但当目标硬件具有多级存储、特殊计算单元及复杂并行约束时，可能的调度组合迅速增加，人工选择策略难以兼顾多种模型和输入形状。

2020年，Zheng等人提出Ansor，利用分层搜索空间、演化搜索和学习型代价模型探索张量程序优化组合，并对网络中的多个优化任务分配搜索资源[8]。其重要贡献在于扩大了可探索的调度空间，使优化不再完全依赖少量人工模板。不过，自动搜索的实际成本还受到候选程序编译、目标设备测量和模型训练开销的影响，不能将离线阶段的搜索成本视为无限可用。

2022年，Zhu等人提出ROLLER，通过与硬件特征相匹配的分块抽象构造候选程序，并使用性能模型减少大量实机测量的需求[9]。该研究体现了另一条路线：利用硬件结构知识约束候选方案，从搜索空间构造阶段降低优化复杂度。对于需要频繁重新编译网络的开发流程，编译耗时与生成程序质量之间的权衡同样值得关注。

上述方法主要面向编译器能够控制算子内部实现的场景。对于一条指令即启动完整卷积或池化操作的NPU，内部乘加阵列调度可能已由硬件固定实现，编译器无法直接改变其循环次序。在此情况下，自动调度的搜索对象应调整为硬件实际暴露的参数，例如输出通道分组、合法分块范围、执行顺序、布局选择以及可用融合模式。将GPU上的线程块调度方法直接移植到此类NPU通常缺少执行基础。

建立代价模型时，也应区分可静态计算的信息与需要硬件测量的信息。张量尺寸、对齐填充量、指令数量和理论访存字节数一般可以由编译结果获得；指令启动延迟、有效带宽及计算与传输的重叠比例则需要通过硬件文档或实验确定。在硬件接口尚不明确时，可以先采用存储占用和传输量作为优化目标，再用板端测量修正性能模型，避免以未经验证的硬件假设支撑优化结论。

3.4 面向资源受限设备的静态存储优化

端侧模型的存储需求包括权重、输入输出、中间激活及临时工作区，其中峰值激活占用与网络拓扑和执行顺序密切相关。逐层分配独立缓冲区虽然实现简单，但会保留大量不再使用的存储空间。静态形状网络能够在编译阶段确定张量大小及使

用关系，因此适合通过生命周期分析复用缓冲区。对于分支网络，还需要考虑共享输入、汇合节点以及最终输出的保留时间。

2021年，Burrello等人提出DORY，将存储层次约束下的分块选择建模为约束规划问题，并生成组织计算和数据传输的C代码[10]。该工作面向使用显式存储管理的低功耗MCU，说明端侧编译器需要将数据搬运作为核心编译对象，而不能仅生成算子调用。其分块与缓冲管理方法依赖目标平台的实际DMA和存储层次条件，因此向另一种NPU迁移时应重新建立硬件约束。

2020年，Lin等人提出MCUNet，通过TinyNAS与TinyEngine协同设计，使网络结构与微控制器上的执行和存储需求相匹配[11]。这一工作将优化范围扩展到模型结构与推理引擎之间，表明部署成本会反过来影响网络设计。对于保持既有YOLO模型不变的编译器研究，可以借鉴其全图存储分析思想，但应将模型结构搜索与编译优化分开评价，避免将更换网络带来的收益归因于编译器。

2021年，Lin等人进一步研究基于局部图像块的推理方法，并通过网络结构与执行调度协同设计形成MCUNetV2[12]。该方法针对网络前部高分辨率特征图导致的存储峰值，通过局部执行降低同时保存的激活规模。其代价是相邻块可能具有重叠感受野，导致额外计算。因此，分块推理既涉及容量问题，也涉及计算重复和边界数据组织问题。

2023年，Zheng等人提出StreamNet，利用流式缓冲减少基于块的推理所产生的冗余计算，并根据设备资源选择相应参数[13]。与简单缩小块尺寸相比，流式处理需要保存可以被后续计算复用的边界数据。该工作提示，存储优化并不是始终追求最小缓冲区，而是需要在缓冲保留、重复计算和执行效率之间进行权衡。

2024年，Chen等人提出MAGIS，研究图变换与图调度的协同存储优化[14]。其方法说明，先独立确定图结构、再固定执行顺序、最后分配地址的流程可能错失跨阶段优化机会。对本课题而言，这一思想可以转化为更小范围的研究问题：在NPU指令和通道布局约束下，联合选择分支执行顺序及缓冲区复用方式，并保留既有拼接别名关系。该问题与通用训练系统的存储优化具有共同基础，但其目标平台、可用操作和约束条件不同。

静态存储规划还需要区分逻辑张量与物理存储对象。两个逻辑张量可能引用同一片连续空间的不同区域，也可能因原位计算共享地址。若只依据张量名称计算生命周期，容易过早释放仍被别名引用的空间。较为稳妥的方法是显式构造存储对象

及其视图，并以全部相关消费者完成使用作为释放条件。对于异步硬件，释放时刻还应根据执行完成事件确定，而非简单依据指令发出次序。

3.5 数据流建模与软硬件协同编译

专用加速器的实际效率受到数据复用方式、存储容量和互连带宽的共同影响。2019年，Kwon等人提出MAESTRO相关研究，通过数据流描述分析DNN计算中的复用、性能和硬件成本[15]。这类模型使不同映射策略能够在统一的硬件约束下进行比较，也说明仅以乘加次数评价网络在加速器上的执行代价并不充分。

2023年，Mei等人提出DeFiNES，统一分析逐层执行与跨层深度优先调度，并将不同存储层次的数据访问纳入代价估计[16]。跨层执行可能减少中间结果的外存访问，但也可能改变权重重复加载次数和片上缓冲需求。因此，评价跨层融合应考虑计算、权重和激活的共同影响，而不能只观察某一类中间张量是否被消除。

2024年，Scherer等人提出Deeploy，面向具有RISC-V多核处理器和NPU的异构微控制器生成优化C代码，并将运行时支持保持在较小规模[17]。该工作与本课题在系统构成上较为接近，说明端侧异构部署需要同时处理硬件算子映射、存储约束及执行组织。其产物形式与直接生成NPU指令镜像不同，但二者都可以将主要决策放在部署前完成。

2025年，Hamdi等人提出MATCH，基于TVM构建面向异构边缘设备的编译方法，通过模型与硬件相关的描述减少为不同目标平台重复开发编译流程的工作[18]。这一研究强调可重定向性：通用图分析与硬件相关的算子实现、存储约束应适当分离。对原型NPU而言，若通道数限制、对齐规则和指令能力被分散写入多个脚本，硬件修改容易造成不一致，因此建立集中、可检查的目标能力描述具有实际意义。

同年，Lopoukhine等人针对具有硬件循环和流式寄存器的RISC-V扩展，研究了多层次编译后端及相应微内核生成方法[19]。该研究表明，硬件扩展只有通过适当的编译抽象被表达和利用，才能转化为软件可获得的性能收益。不过，这类细粒度指令扩展与完整算子级NPU指令在控制能力上不同，二者的优化方法不能直接等同。

宏指令并不意味着没有编译优化空间，而是将部分细粒度执行选择固化在硬件中。在已有指令集下，编译器仍可以优化算子组合、分支顺序、通道分组、数据布局和缓冲区复用；如果允许修改硬件，则可以研究后处理融合、可配置输出步长或

显式数据搬运等能力。新增能力应由实际瓶颈驱动，并同时报告硬件资源、时序和软件收益，使软硬件协同优化具有可评价的边界。

3.6 数值一致性与编译结果验证

AI编译器的正确性不仅体现为程序可以执行，还体现为编译后计算符合约定的数值语义。2018年，Jacob等人研究了整数算术推理所需的量化与训练方法，明确了量化表示与高效整数执行之间的关系[20]。对于定点NPU，权重格式、偏置尺度、累加位宽、移位和饱和方式都会影响结果。图变换若改变这些操作的相对位置，即使浮点表达式保持等价，也可能产生不同输出。

因此，验证体系宜从单算子、典型子图和完整网络三个层次建立。单算子测试用于覆盖边界填充、奇数尺寸、非对齐通道和极端数值；子图测试用于检查分支、拼接和融合变换；完整网络测试则用于检查端到端功能及任务指标。二进制输出稳定或回归文件一致可以证明生成过程的可重复性，但不能独立证明网络数值正确。

对于已经能够运行YOLO网络的FPGA系统，识别结果可以作为端到端功能验证的重要证据，但还需要结合逐层输出比对及明确数据集上的任务指标评估误差。如果缺少完整数值参考模型，应首先记录硬件运算约定，再建立与之匹配的软件参考实现。这样既能够验证现有编译流程，也能够为后续存储复用和指令融合提供稳定的回归依据。

3.7 国内外研究比较与发展趋势

从研究组织形式看，AI编译器成果具有明显的跨机构合作特点。国内研究团队与国际高校、企业在通用编译框架、自动调度和存储优化方面共同推进相关研究。与简单按地区划分相比，按照研究对象、可控制的执行层次和硬件约束比较不同工作，更有助于判断其与具体课题的关系。表1归纳了具有代表性的研究方向及其对宏指令NPU的启示。

表1 代表性研究与宏指令NPU编译的关系

研究方向	代表工作及主要方法	对本课题的启示与适用边界
编译表示	TVM、MLIR、TensorIR：分层表示与目标映射	保留图语义和硬件约束；按需求控制实现规模

研究方向	代表工作及主要方法	对本课题的启示与适用边界
图结构优化	TASO、DNNFusion：图替换与融合	需核对定点语义；软件分组不等于消除硬件访存
自动调度	Ansor、ROLLER：搜索或构造有效程序	仅优化硬件实际暴露的分组、布局与调度参数
MCU部署	DORY、MCUNet：分块、存储与执行协同	借鉴资源建模；DMA、计算内核和目标硬件不同
跨层存储	MCUNetV2、StreamNet、MAGIS：分块、流式与联合规划	综合考虑空间、重算和别名；不直接搬用训练场景结论
异构编译	Deeploy、MATCH：异构映射与静态代码生成	目标与本课题接近；C代码与NPU指令镜像形式不同
硬件代价	MAESTRO、DeFiNES：数据流及层次化访问建模	先明确存储和执行能力，再评估跨层或指令扩展

现有研究呈现出三个较为明确的趋势。首先，优化层次逐渐由单算子扩展至图结构、调度和存储的协同决策，编译阶段之间的边界不再只是固定流水线。其次，端侧研究越来越重视可部署性，将内存预算、数值格式与运行支持作为设计约束。最后，硬件能力的结构化描述与测量反馈逐步进入编译流程，使编译器能够解释映射决策并适应目标平台的变化。

对于本课题，值得进一步研究的不是重新证明离线编译可行，而是在已有部署基础上明确尚未解决的优化问题。现有通用方法为生命周期分析和存储复用提供了基础，课题需要进一步说明通道对齐、输出地址约束、拼接别名及粗粒度执行对优化造成的特殊影响。只有将这些约束纳入算法，并通过与常规静态分配及独立阶段优化的比较证明作用，才能建立具有针对性的研究贡献。

4.结论

本文围绕面向端侧异构系统的AI编译器，梳理了多层次中间表示、计算图重写、张量自动调度、静态存储优化及软硬件协同编译等方面的研究进展。相关工作表明，AI编译器已经形成从模型语义到目标执行的多层优化体系，离线编译及轻量运行支持也是资源受限设备上的成熟技术方向。因此，完成模型到硬件的部署流程是重要的工程基础，但需要进一步形成可解释、可复现的优化方法，才能体现论文研究的学术价值。

面向RISC-VSoC中的宏指令NPU，后续研究可以将布局约束下的张量生命周期分析、分支执行顺序选择和存储地址复用作为主要切入点，在不改变网络功能的前提下降低峰值工作区需求。对于硬件支持明确且出现频繁的计算模式，可以进一步评估后处理融合或输出步长扩展的收益。研究应以明确的约束模型和对照实验为基础，将功能验证、数值一致性、性能收益及硬件代价分别报告，从而形成完整的软硬件协同研究结论。

5.主要参考文献

[1] Li M, Liu Y, Liu X, et al. The Deep Learning Compiler: A Comprehensive Survey[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 32(3): 708-727. DOI: 10.1109/TPDS.2020.3030548.

[2] Chen T, Moreau T, Jiang Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning[A]. In: 13th USENIX Symposium on Operating Systems Design and Implementation[C]. Berkeley: USENIX Association, 2018: 578-594. https://www.usenix.org/conference/osdi18/presentation/chen.

[3] Lattner C, Amini M, Bondhugula U, et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation[A]. In: 2021 IEEE/ACM International Symposium on Code Generation and Optimization[C]. Piscataway: IEEE, 2021: 2-14. DOI: 10.1109/CGO51591.2021.9370308.

[4] Ragan-Kelley J, Barnes C, Adams A, et al. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines[A]. In: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation[C]. New York: ACM, 2013: 519-530. DOI: 10.1145/2491956.2462176.

[5] Feng S, Hou B, Jin H, et al. TensorIR: An Abstraction for Automatic Tensorized Program Optimization[A]. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2[C]. New York: ACM, 2023: 804-817. DOI: 10.1145/3575693.3576933.

[6] Jia Z, Padon O, Thomas J, et al. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions[A]. In: Proceedings of the 27th ACM Symposium on Operating Systems Principles[C]. New York: ACM, 2019: 47-62. DOI: 10.1145/3341301.3359630.

[7] Niu W, Guan J, Wang Y, et al. DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion[A]. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation[C]. New York: ACM, 2021: 883-898. DOI: 10.1145/3453483.3454083.

[8] Zheng L, Jia C, Sun M, et al. Ansor: Generating High-Performance Tensor Programs for Deep Learning[A]. In: 14th USENIX Symposium on Operating Systems Design and Implementation[C]. Berkeley: USENIX Association, 2020: 863-879. https://www.usenix.org/conference/osdi20/presentation/zheng.

[9] Zhu H, Wu R, Diao Y, et al. ROLLER: Fast and Efficient Tensor Compilation for Deep Learning[A]. In: 16th USENIX Symposium on Operating Systems Design and Implementation[C]. Berkeley: USENIX Association, 2022: 233-248. https://www.usenix.org/conference/osdi22/presentation/zhu.

[10] Burrello A, Garofalo A, Bruschi N, et al. DORY: Automatic End-to-End Deployment of Real-World DNNs on Low-Cost IoT MCUs[J]. IEEE Transactions on Computers, 2021, 70(8): 1253-1268. DOI: 10.1109/TC.2021.3066883.

[11] Lin J, Chen W M, Lin Y, et al. MCUNet: Tiny Deep Learning on IoT Devices[A]. In: Advances in Neural Information Processing Systems 33[C]. Red Hook: Curran Associates, 2020: 11711-11722.

[12] Lin J, Chen W M, Cai H, et al. Memory-Efficient Patch-Based Inference for Tiny Deep Learning[A]. In: Advances in Neural Information Processing Systems 34[C]. Red Hook: Curran Associates, 2021: 2346-2358.

[13] Zheng H S, Liu Y Y, Hsu C F, et al. StreamNet: Memory-Efficient Streaming Tiny Deep Learning Inference on the Microcontroller[A]. In: Advances in Neural Information Processing Systems 36[C]. Red Hook: Curran Associates, 2023: 37160-37172. DOI: 10.52202/075280-1615.

[14] Chen R, Ding Z, Zheng S, et al. MAGIS: Memory Optimization via Coordinated Graph Transformation and Scheduling for DNN[A]. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3[C]. New York: ACM, 2024: 607-621. DOI: 10.1145/3620666.3651330.

[15] Kwon H, Chatarasi P, Pellauer M, et al. Understanding Reuse, Performance, and Hardware Cost of DNN Dataflow[A]. In: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture[C]. New York: ACM, 2019: 754-768. DOI: 10.1145/3352460.3358252.

[16] Mei L, Goetschalckx K, Symons A, et al. DeFiNES: Enabling Fast Exploration of the Depth-First Scheduling Space for DNN Accelerators through Analytical Modeling[A]. In: 2023 IEEE International Symposium on High-Performance Computer Architecture[C]. Piscataway: IEEE, 2023: 570-583. DOI: 10.1109/HPCA56546.2023.10071098.

[17] Scherer M, Macan L, Jung V J B, et al. Deeploy: Enabling Energy-Efficient Deployment of Small Language Models on Heterogeneous Microcontrollers[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2024, 43(11): 4009-4020. DOI: 10.1109/TCAD.2024.3443718.

[18] Hamdi M A, Daghero F, Sarda G M, et al. MATCH: Model-Aware TVM-Based Compilation for Heterogeneous Edge Devices[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2025, 44(10): 3844-3857. DOI: 10.1109/TCAD.2025.3556967.

[19] Lopoukhine A, Ficarelli F, Vasiladiotis C, et al. A Multi-Level Compiler Backend for Accelerated Micro-Kernels Targeting RISC-V ISA Extensions[A]. In: Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization[C]. New York: ACM, 2025: 163-178. DOI: 10.1145/3696443.3708952.

[20] Jacob B, Kligys S, Chen B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[A]. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition[C]. Piscataway: IEEE, 2018: 2704-2713. DOI: 10.1109/CVPR.2018.00286.

附录 文献来源与等级核对

本综述共引用20篇正式发表的学术文献。来源等级统一按《中国计算机学会推荐国际学术会议和期刊目录（2022年）》核对，其中18篇来自A类、2篇来自B类；不将arXiv预印本单独作为等级依据，也不混用未注明版本的SCI分区。以下为检索核对信息，文献著录以正式出版记录为准。

附表1 引用文献发表来源核对

正式发表来源	CCF等级（2022版）	本文文献序号
TPDS	A	[1]
OSDI	A	[2], [8], [9]
CGO	B	[3], [19]
PLDI	A	[4], [7]
ASPLOS	A	[5], [14]
SOSP	A	[6]
TC	A	[10]
NeurIPS	A	[11], [12], [13]
MICRO	A	[15]
HPCA	A	[16]
TCAD	A	[17], [18]
CVPR	A	[20]

目录来源：https://www.ccf.org.cn/Academic_Evaluation/By_category/。出版信息核对渠道包括ACM、IEEE、USENIX与NeurIPS正式论文集、出版DOI及作者机构公开版本。检索整理日期：2026年10月16日。